SOFTWARE DESIGN SPECIFICATION

Academic Project Planning and Automated Architecture Design Tool

Name :- Akhya Sinha
Reg.No.:- 23BKT0045
Date :- 27th February 2026

CONTENT

1 Introduction 1
1.1 Purpose 1
1.2 Scope 1
1.3 Definitions and Acronyms 2
1.4 References 2

2 Decomposition Description 3
2.1 Module Decomposition 3
2.2 Concurrent Process Decomposition 4
2.3 Data Decomposition 5

3 Dependency Description 6
3.1 Inter-module Dependencies 6
3.1.1 Independent Modules 6
3.1.2 Dependent Modules 7
3.2 Inter-process Dependencies 7
3.3 Data Dependencies 8

4 Interface Description 9
4.1 Module Interface 9
4.1.1 Project Input Module Description 9
4.1.2 Input Validation Module Description 11
4.1.3 Rule Engine Module Description 13
4.1.4 Project Planning Module Description 15
4.1.5 Architecture Recommendation Module Description 17
4.1.6 Output Generation Module Description 19
4.1.7 System Administrator Module Description 21

4.2 Process Interface 23
4.2.1 Project Input Process Description 23
4.2.2 Rule Evaluation Process Description 24
4.2.3 Input Validation Process Description 25

4.2.4 Project Planning Process Description 26
4.2.5 Architecture Generation Process Description 27
4.2.6 Output Display Process Description 28

5 Detailed Design 29
5.1 Module Detailed Design 29
5.1.1 System Interface Provider 29
5.1.2 Project Input Module Detailed Design 31
5.1.3 Input Validation Module Detailed Design 33
5.1.4 Rule Engine Module Detailed Design 35
5.1.5 Architecture Recommendation Module Detailed Design 37
5.1.6 Output Generation Module Detailed Design 39

Appendix A — Academic Project Planning Tool: Class Diagram 41
Appendix B — UML Diagram Design Reference for Modelio 42

List of Figures

Figure 1. Data Flow Diagram
Figure 2. Project Input Module, Input Form Screen
Figure 3. Project Input Module, Project Details Entry Screen
Figure 4. Rule Engine Module, Rule Evaluation Screen
Figure 5. Rule Engine Module, Rule Matching Screen
Figure 6. Input Validation Module, Validation Feedback Screen
Figure 7. Input Validation Module, Error Display Screen
Figure 8. Project Planning Module, Phase Plan Generation Screen
Figure 9. Project Planning Module, Phase Plan Display Screen
Figure 10. Architecture Recommendation Module, Architecture Output Screen
Figure 11. Architecture Recommendation Module, Architecture Details Screen
Figure 12. Output Generation Module, Final Output Screen
Figure 13. Output Generation Module, Export Options Screen
Figure 14. System Administrator Module, System Management Screen
Figure 15. Project Input Module Class Diagram
Figure 16. Rule Engine Module Class Diagram
Figure 17. Input Validation Module Class Diagram
Figure 18. Project Planning Module Class Diagram
Figure 19. Architecture Recommendation Module Class Diagram
Figure 20. Output Generation Module Class Diagram

List of Tables

Table of Definitions, Acronyms, and Abbreviations
Table of References
Table of Module Dependencies
Table of Use Case — Module Interface Mapping
Table of Module Input/Output Specifications
Table of Detailed Design — Data Members and Methods

1 Introduction

1.1 Purpose

This Software Design Specification (SDS) document describes the comprehensive design of the Academic Project Planning and Automated Architecture Design Tool. The purpose of this document is to provide a complete, implementation-ready design reference that translates all functional and non-functional requirements defined in the Software Requirements Specification (SRS) into an actionable architectural and component-level design blueprint.

This document defines the system's modular decomposition, inter-module dependencies, interface specifications, process descriptions, and detailed component designs — including data members, methods, and interaction patterns. It serves as the primary engineering reference for software developers during the implementation phase, for evaluators verifying design completeness, and for future maintainers seeking to enhance or extend the system.

1.2 Scope

The Academic Project Planning and Automated Architecture Design Tool is a standalone, rule-based decision-support system intended exclusively for academic and laboratory use. The scope of this SDS covers the complete software design of all system modules, including the Project Input Module, Input Validation Module, Rule Engine Module, Project Planning Module, Architecture Recommendation Module, and Output Generation Module.

The system accepts basic project parameters — project type (mini project or final-year project), team size, total project duration in weeks, and technology stack — and processes them through a layered three-tier architecture comprising a Presentation Layer, Application Layer, and Data Layer. The system generates two key outputs: a phase-wise project plan with allocated durations, and a high-level software architecture recommendation with supporting rationale.

1.3 Definitions and Acronyms

Definition / Acronym	Description
API	Application Programming Interface — a defined interface for software component interaction
Three-Tier Architecture	A layered architectural pattern separating Presentation, Application Logic, and Data concerns
Rule-Based Engine	A system component that evaluates predefined logical rules to produce deterministic outputs

Monolithic Architecture	A software pattern where all components are tightly coupled and deployed as a single unit
MVC	Model-View-Controller — a design pattern separating data, presentation, and control logic
Phase-wise Plan	A structured project schedule dividing duration across standard SDLC phases
Module	A cohesive, independently deployable unit of the system with defined responsibilities
Interface	The defined contract through which modules exchange data and control signals
Dependency	A relationship where one module requires the services or data of another to function
Decomposition	The process of dividing a system into smaller, manageable functional units
DFD	Data Flow Diagram — a graphical representation of data movement through a system
ERD	Entity Relationship Diagram — a diagram showing entities and their relationships
RTM	Requirements Traceability Matrix — a mapping of requirements to design elements

1.4 References

Reference	**Description**
[IEEE-1] IEEE Std 1016-2009. IEEE Standard for Information Technology — Systems Design — Software Design Descriptions.	Governing standard for the structure, content, and notation used in this SDS document.
[SOM-1] I. Sommerville. Software Engineering, 10th ed., Pearson Education, 2016.	Foundational reference for software design principles, modular decomposition, and component design methodology.
[KRUC-1] P. Kruchten. 'Architectural Blueprints — The 4+1 View Model of Software Architecture,' IEEE Software, vol. 12, no. 6, 1995.	Reference for multi-view architectural design and component interaction modelling.
[WIEG-1] K. Wiegers & J. Beatty. Software Requirements, 3rd ed., Microsoft Press, 2013.	Reference for requirements-to-design translation methodology and interface specification.

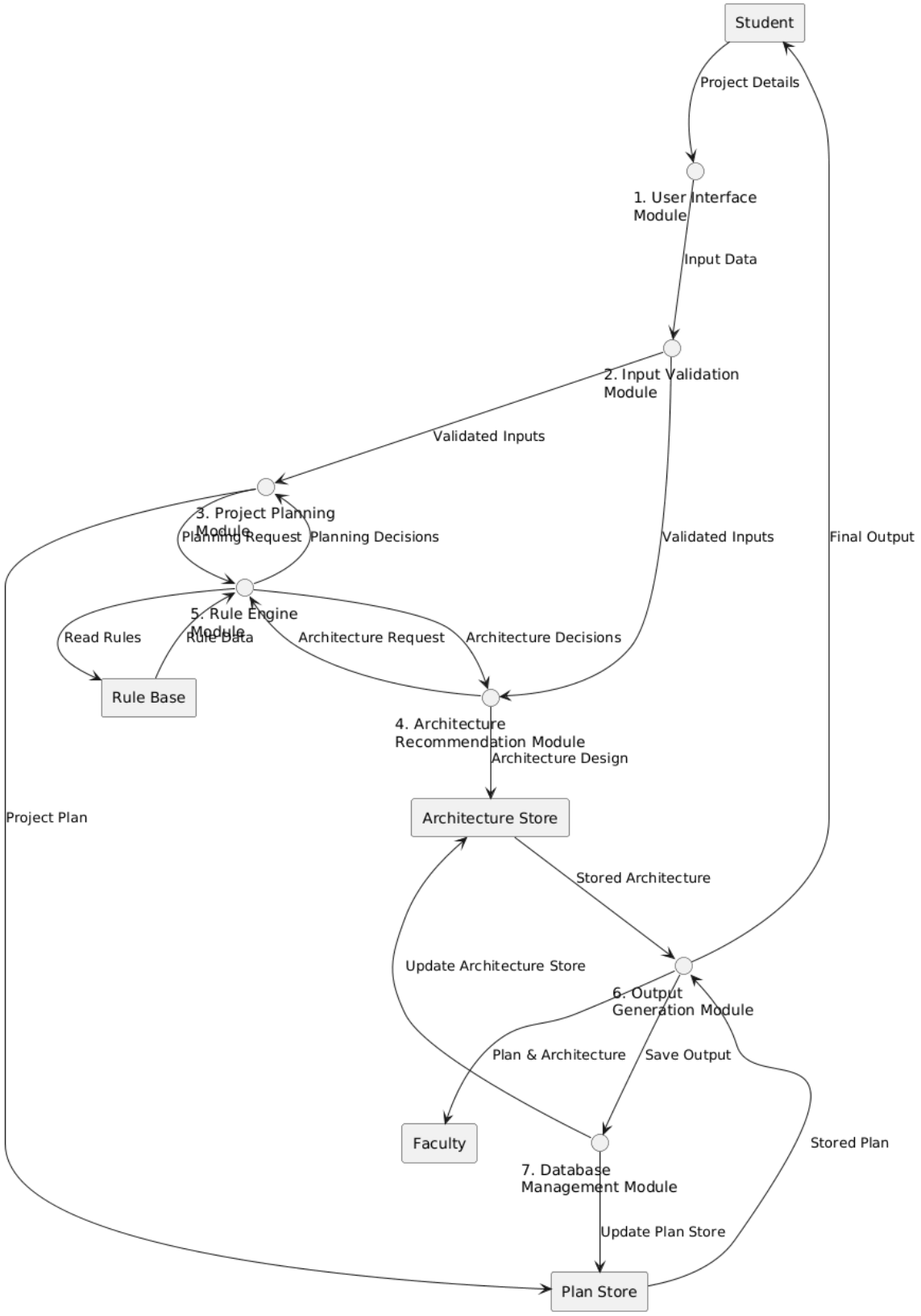
2 Decomposition Description

This section describes the decomposition of the Academic Project Planning and Automated Architecture Design Tool into its principal modules, concurrent processes, and data structures.

2.1 Module Decomposition

The system is decomposed into six primary functional modules and one administrative module. Each module encapsulates a distinct responsibility and exposes a defined interface to the other modules. The module decomposition is grounded in the Three-Tier Architecture — modules are grouped according to the Presentation, Application, and Data layers they belong to.

Module ID	Module Name	Functional Responsibility
MOD-01	Project Input Module	Provides the user-facing interface for collecting project parameters. Captures project type, team size, project duration, and technology stack from the student. Belongs to the Presentation Layer.
MOD-02	Input Validation Module	Validates all user-provided inputs for completeness, correctness, and value range conformance before any processing is initiated. Acts as the Application Layer gatekeeper.
MOD-03	Rule Engine Module	Core decision-making component. Evaluates validated inputs against predefined planning rules and architecture rules stored in the Rule Base. Produces planning and architecture decisions. Application Layer.
MOD-04	Project Planning Module	Receives planning decisions from the Rule Engine and generates a structured, phase-wise project plan with duration allocations across standard SDLC phases. Application Layer.
MOD-05	Architecture Recommendation Module	Receives architecture decisions from the Rule Engine and constructs a complete architecture recommendation object with type, description, components, and rationale. Application Layer.
MOD-06	Output Generation Module	Assembles, formats, and presents the combined project plan and architecture recommendation to the user in a clear, structured output screen. Presentation Layer.
MOD-07	System Administrator Module	Provides administrative functionality for managing the Rule Base — adding, modifying, and removing planning and architecture rules. Data Layer management interface.



SOFTWARE USED :- MODELIO

2.2 Concurrent Process Decomposition

The Academic Project Planning Tool decomposes its runtime behavior into several concurrent processes that execute in response to user events. Although the current version is designed as a single-user, sequential application, the process decomposition supports future extension to multi-user, concurrent operation as described in SRS Section 2.9.5.

Process ID	Process Name	Description
PROC-01	Input Collection Process	Runs continuously while the application is in the InputEntry state, accepting field-level input events from the student. Operates in the Presentation Layer and communicates asynchronously with the Validation Process upon form submission.
PROC-02	Input Validation Process	Triggered by form submission. Validates all input fields concurrently and aggregates error results before returning a pass or fail decision. Supports parallel field-level checks for null, type, and range constraints.
PROC-03	Rule Evaluation Process	Triggered by successful validation. Evaluates the rule base to find matching planning and architecture rules for the given input set. Planning and architecture rule evaluation can proceed in parallel as they are independent decision paths.
PROC-04	Plan Generation Process	Triggered by a matching planning rule. Calculates phase durations and builds the project plan object. Runs independently of the Architecture Generation Process.
PROC-05	Architecture Generation Process	Triggered by a matching architecture rule. Builds the architecture recommendation object with description, components, and rationale. Runs independently of the Plan Generation Process.
PROC-06	Output Rendering Process	Triggered after both Plan and Architecture Generation Processes complete. Assembles and renders the combined output to the student's screen.
PROC-07	System Administration Process	Runs in a separate administrative session. Manages the Rule Base through CRUD operations on planning and architecture rules. Independent of the student-facing processes.

2.3 Data Decomposition

The system's data is decomposed into discrete data entities, each owned by a specific layer and accessed by defined modules. The data decomposition reflects the separation of concerns across the Three-Tier Architecture.

Data Entity	Owner Layer	Description
Project Input Data	Presentation Layer	Raw user-provided values: project_type (String), team_size (Integer), duration_weeks (Integer), technology (String). Created by MOD-01, consumed by MOD-02.
Validated Input Data	Application Layer	Verified copy of Project Input Data after all validation checks pass. Produced by MOD-02, consumed by MOD-03.
Validation Error Data	Presentation Layer	Structured error list returned when validation fails. Contains field name and error description pairs. Produced by MOD-02, rendered by MOD-01.
Planning Rule Data	Data Layer	Predefined rule records mapping input conditions to phase allocation percentages. Stored in the Rule Base. Read by MOD-03.
Architecture Rule Data	Data Layer	Predefined rule records mapping input conditions to architecture type decisions. Stored in the Rule Base. Read by MOD-03.
Planning Decision Data	Application Layer	Rule evaluation result specifying phase allocation percentages. Produced by MOD-03, consumed by MOD-04.
Architecture Decision Data	Application Layer	Rule evaluation result specifying the recommended architecture type. Produced by MOD-03, consumed by MOD-05.
Project Plan Data	Data Layer	Generated phase-wise project plan object. Contains plan_id, phases list, and timestamps. Produced by MOD-04, stored in Plan Store, read by MOD-06.
Architecture Design Data	Data Layer	Generated architecture recommendation object. Contains architecture_type, description, components, and rationale. Produced by MOD-05, stored in Architecture Store, read by MOD-06.
System Output Data	Presentation Layer	Compiled, formatted display data combining plan and architecture outputs. Produced by MOD-06, rendered to the student's screen.

3 Dependency Description

This section specifies the inter-module dependencies, inter-process dependencies, and data dependencies of the Academic Project Planning and Automated Architecture Design Tool. Understanding these dependencies is essential for determining module build order, testing strategy, and impact analysis for future changes.

3.1 Inter-module Dependencies

The inter-module dependencies define the functional relationships between the system's modules. A dependency exists when Module A requires the services, outputs, or interfaces of Module B to fulfil its own responsibility. The dependency graph follows a strict layered direction: Presentation → Application → Data, with no upward dependencies.

3.1.1 Independent Modules

Independent modules are those that have no runtime dependencies on other system modules. They provide fundamental services consumed by higher-level modules and can be developed, tested, and deployed in isolation.

Module	**Independence Rationale**	**Services Provided**
MOD-07: System Administrator Module	Operates in a separate administrative session with no dependencies on student-facing modules. Communicates only with the Rule Base data store.	Rule Base management — add, update, delete, and list planning and architecture rules.
Rule Base (Data Store)	A passive data store with no active dependencies. Stores predefined rules and responds to read/write requests.	Provides planning rule records and architecture rule records to MOD-03 on request.
Plan Store (Data Store)	A passive in-memory store with no module dependencies.	Temporarily persists generated project plan objects for retrieval by MOD-06.
Architecture Store (Data Store)	A passive in-memory store with no module dependencies.	Temporarily persists generated architecture recommendation objects for retrieval by MOD-06.

3.1.2 Dependent Modules

Dependent modules are those that rely on the outputs or interfaces of one or more other modules. The following table lists all dependent modules and their specific dependencies.

Module	Depends On	Nature of Dependency
MOD-01: Project Input Module	MOD-02 (Validation Module)	MOD-01 submits collected input data to MOD-02 for validation. It receives either validated data (proceed) or a structured error list (display errors). Functional dependency — cannot proceed without validation result.
MOD-02: Input Validation Module	MOD-03 (Rule Engine Module)	MOD-02 forwards validated input data to MOD-03 for rule evaluation. Functional dependency — validation output is the required input for rule processing.
MOD-03: Rule Engine Module	Rule Base (Data Layer), MOD-04, MOD-05	MOD-03 reads planning and architecture rules from the Rule Base (data dependency). It then invokes MOD-04 and MOD-05 with the resulting decisions (functional dependency).
MOD-04: Project Planning Module	MOD-03 (Rule Engine), Plan Store	MOD-04 requires a planning decision from MOD-03 (functional). It writes the generated plan to the Plan Store (data dependency).
MOD-05: Architecture Recommendation Module	MOD-03 (Rule Engine), Architecture Store	MOD-05 requires an architecture decision from MOD-03 (functional). It writes the recommendation to the Architecture Store (data dependency).
MOD-06: Output Generation Module	Plan Store, Architecture Store	MOD-06 reads the generated plan from the Plan Store and the architecture recommendation from the Architecture Store. Pure data dependency — no functional dependency on MOD-04 or MOD-05 directly.

3.2 Inter-process Dependencies

The following table specifies the execution dependencies between the system's runtime processes. An inter-process dependency exists when Process B cannot begin execution until Process A has successfully completed.

Process	Depends On	Dependency Description
PROC-02: Input Validation	PROC-01: Input Collection	Validation process cannot execute until the student completes and submits the input form. Triggered by form submission event.

PROC-03: Rule Evaluation	PROC-02: Input Validation	Rule evaluation requires a clean set of validated input data. Cannot proceed if validation returns errors.
PROC-04: Plan Generation	PROC-03: Rule Evaluation	Plan generation requires a planning decision from the rule evaluation result. Strict sequential dependency.
PROC-05: Architecture Generation	PROC-03: Rule Evaluation	Architecture generation requires an architecture decision from the rule evaluation result. Can execute in parallel with PROC-04 once PROC-03 is complete.
PROC-06: Output Rendering	PROC-04 AND PROC-05	Output rendering requires both the project plan (from PROC-04) and the architecture recommendation (from PROC-05) to be complete before assembling the display.

3.3 Data Dependencies

Data dependencies describe which modules produce and consume specific data entities. A data dependency exists when Module B requires data that is produced or managed by Module A.

Data Entity	**Producer Module**	**Consumer Module(s)**
Project Input Data	MOD-01 (Project Input Module)	MOD-02 (Input Validation Module)
Validated Input Data	MOD-02 (Input Validation Module)	MOD-03 (Rule Engine Module)
Validation Error Data	MOD-02 (Input Validation Module)	MOD-01 (for error display in UI)
Planning Rule Data	Rule Base / MOD-07 (System Admin)	MOD-03 (Rule Engine Module)
Architecture Rule Data	Rule Base / MOD-07 (System Admin)	MOD-03 (Rule Engine Module)
Planning Decision Data	MOD-03 (Rule Engine Module)	MOD-04 (Project Planning Module)
Architecture Decision Data	MOD-03 (Rule Engine Module)	MOD-05 (Architecture Recommendation Module)
Project Plan Data	MOD-04 (Project Planning Module)	Plan Store → MOD-06 (Output Generation Module)
Architecture Design Data	MOD-05 (Architecture Recommendation Module)	Architecture Store → MOD-06 (Output Generation Module)
System Output Data	MOD-06 (Output Generation Module)	Student (via Presentation Layer UI)

4 Interface Description

This section provides complete interface specifications for all modules of the Academic Project Planning and Automated Architecture Design Tool. The interface description covers both Module Interfaces (the data and control contracts between modules) and Process Interfaces (the stimulus-response behaviour of each system process from the user's perspective).

4.1 Module Interface

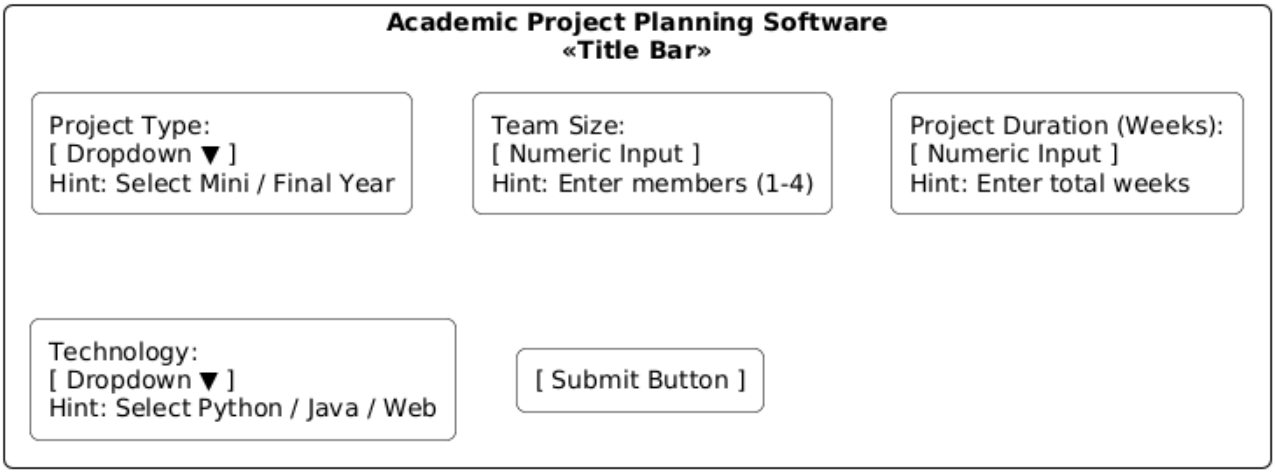
The module interface section specifies the inputs, outputs, and functional responsibilities of each system module as seen from the perspective of external callers. Each module interface acts as a contract that must be honoured regardless of the internal implementation.

4.1.1 Project Input Module Description

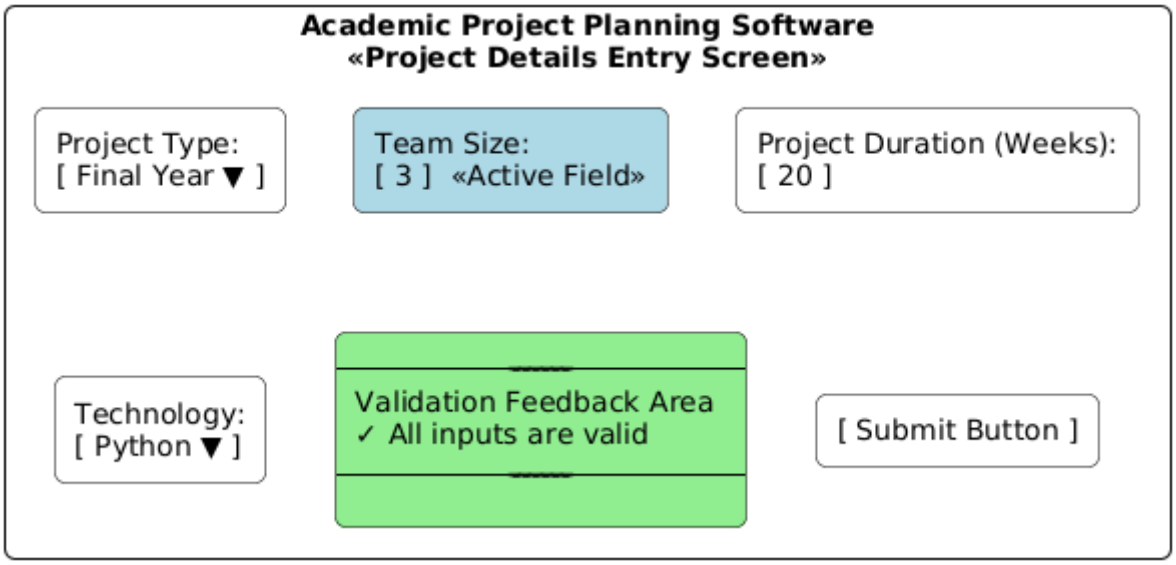
The Project Input Module (MOD-01) serves as the primary user-facing interface of the system. It is responsible for presenting the input form, collecting project parameter values from the student, and submitting them to the Input Validation Module. This module belongs to the Presentation Layer and has no knowledge of the processing or data layers.

Module Name	Project Input Module (MOD-01)
Layer	Presentation Layer
Interface Type	User Interface — Event-Driven
Input Received	User events: field value entry (project_type, team_size, duration_weeks, technology), form submission trigger (Submit button click), and modification requests (Modify Inputs button click)
Output Produced	• Validated: Forwards collected input dictionary {project_type, team_size, duration_weeks, technology} to MOD-02 • Errors: Renders inline error messages adjacent to failed input fields • Output: Renders the final formatted output received from MOD-06 on the Output Screen
Use Cases Supported	UC-05: Enter Project Details | UC-01: Modify Project Inputs | UC-02: View Output
Actors	Student

Error Handling	Displays field-level error messages with clear descriptions. Pre-fills form with previously entered values. Does not clear valid fields when errors are shown for other fields.
Constraints	Must provide both console-based and web-based interface options (SRS Constraint 8). Must be usable by students with basic computer knowledge (SRS Constraint 7).



SOFTWARE USED :- MODELIO

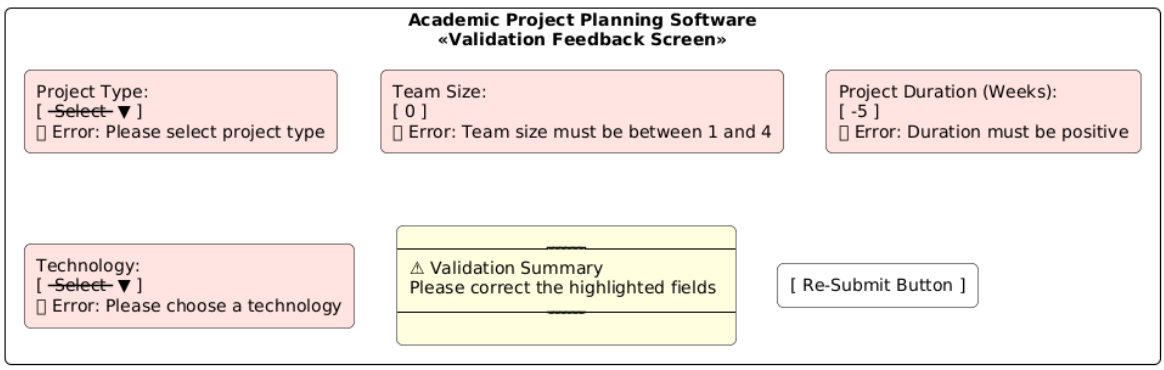


SOFTWARE USED :- MODELIO

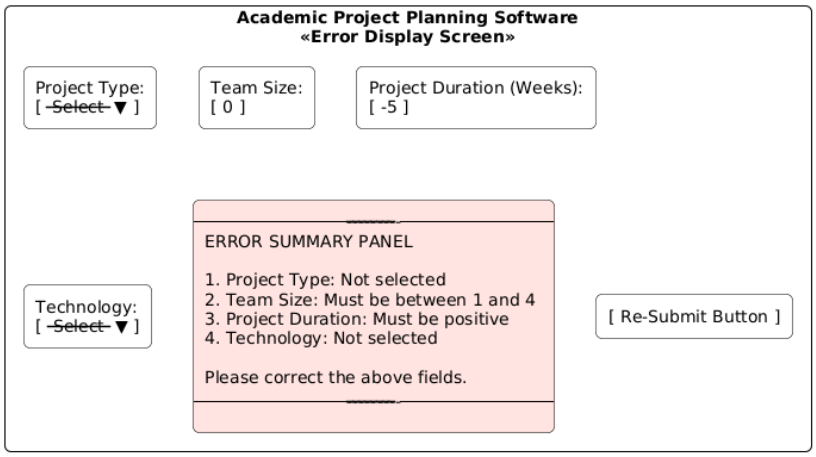
4.1.2 Input Validation Module Description

The Input Validation Module (MOD-02) acts as the gatekeeper of the Application Layer. It receives raw input data from MOD-01 and applies a series of validation checks to ensure that all field values are present, correctly typed, and within the acceptable ranges defined by the system constraints. Only after all checks pass does the module forward data to the Rule Engine Module.

Module Name	Input Validation Module (MOD-02)
Layer	Application Layer
Interface Type	Service Interface — Synchronous
Input Received	Raw input dictionary from MOD-01: {project_type: String, team_size: Integer, duration_weeks: Integer, technology: String}
Output Produced	• Success Path: Returns validated input dictionary to MOD-03 (Rule Engine Module) • Failure Path: Returns structured error list [{field: String, error_message: String}] to MOD-01 for display
Use Cases Supported	UC-05: Enter Project Details (validation sub-step)
Validation Rules Applied	1. Null/empty check: All four fields must be non-null and non-empty. 2. Enum check: project_type ∈ {mini_project, final_year_project} 3. Enum check: technology ∈ {web, mobile, desktop, data_science, embedded} 4. Range check: team_size ∈ [1, 10] (Integer) 5. Range check: duration_weeks ∈ [4, 52] (Integer)
Error Handling	All validation errors are collected in a single pass before returning. The module returns all identified errors simultaneously rather than stopping at the first failure, enabling the UI to display all issues at once.



SOFTWARE USED :- MODELIO

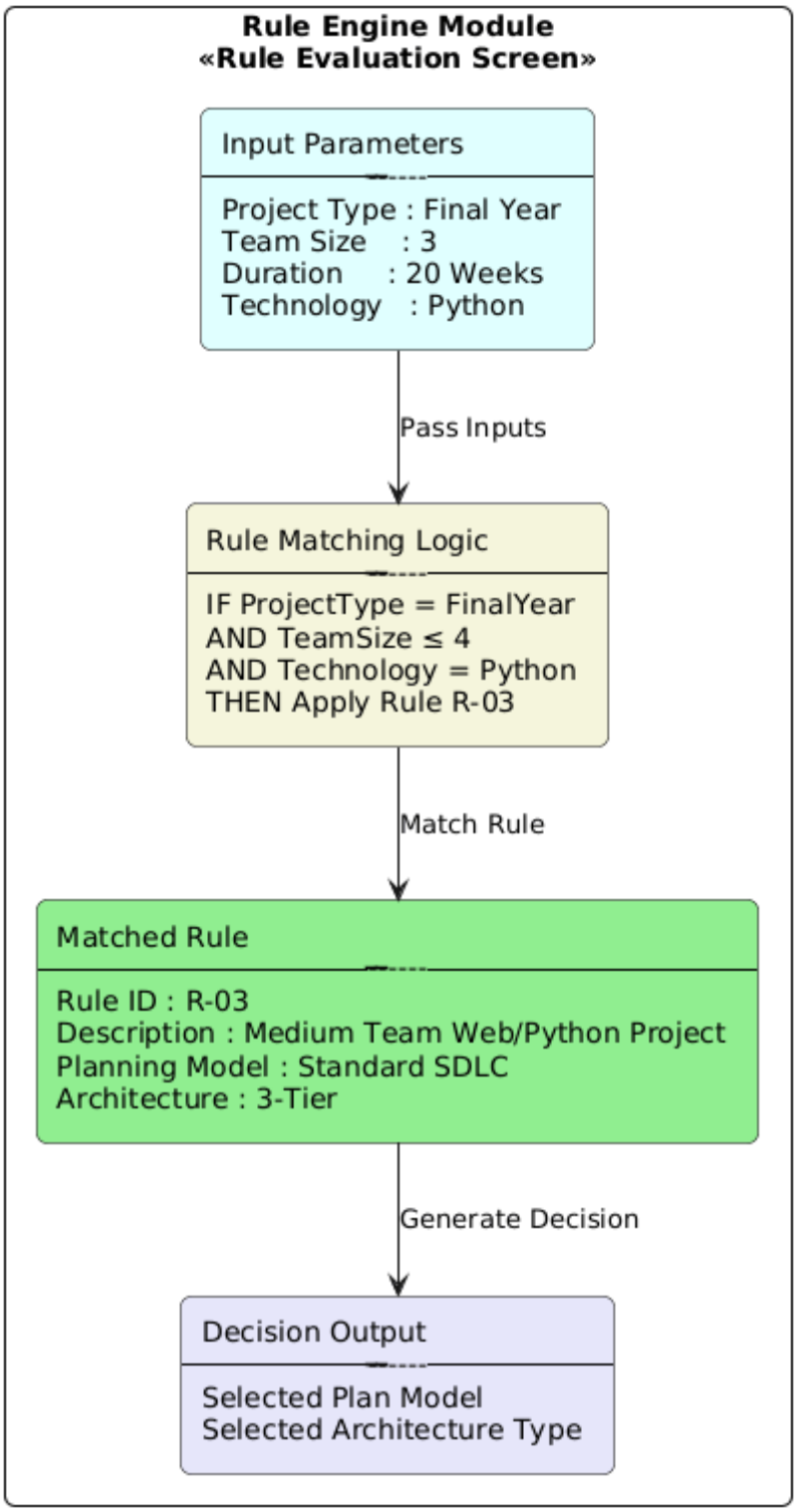


SOFTWARE USED :- MODELIO

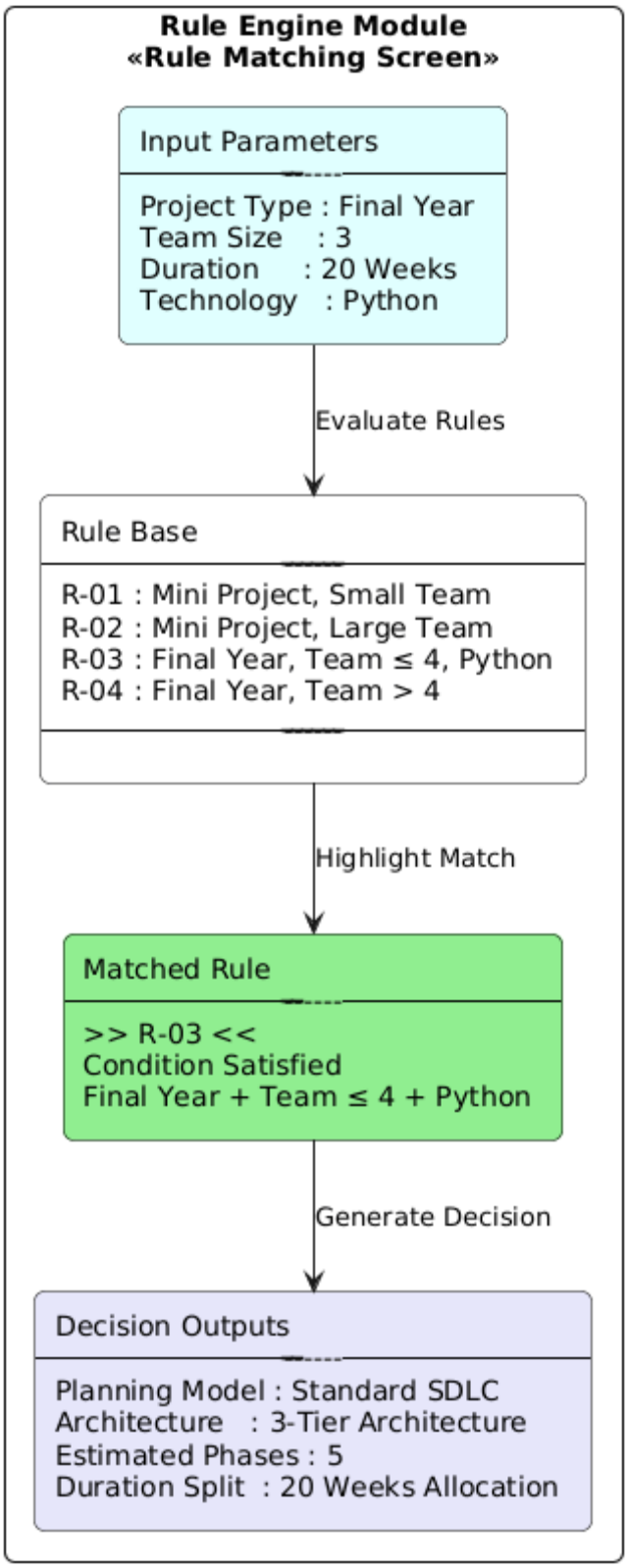
4.1.3 Rule Engine Module Description

The Rule Engine Module (MOD-03) is the core intelligence component of the Academic Project Planning Tool. It receives validated project inputs and evaluates them against a pre-configured set of planning rules and architecture rules stored in the Rule Base. The module produces two decisions: a planning decision (which phase allocation profile to apply) and an architecture decision (which architectural style to recommend). These decisions are dispatched to MOD-04 and MOD-05 respectively.

Module Name	Rule Engine Module (MOD-03)
Layer	Application Layer
Interface Type	Service Interface — Synchronous (Rule Evaluation)
Input Received	Validated input dictionary from MOD-02: {project_type, team_size, duration_weeks, technology}
Output Produced	• Planning Decision: {rule_id, phase_allocations: [{phase_name, percentage}]} → dispatched to MOD-04 • Architecture Decision: {rule_id, architecture_type, rationale} → dispatched to MOD-05
Use Cases Supported	UC-03: Generate Architecture Design | UC-04: Generate Project Plan
Rule Evaluation Logic	1. Load all planning rules from Rule Base (ordered by priority) 2. Evaluate each rule condition against validated inputs 3. Select first matching planning rule; extract phase allocation action 4. Load all architecture rules from Rule Base (ordered by priority) 5. Evaluate each rule condition against validated inputs 6. Select first matching architecture rule; extract architecture type action 7. Dispatch both decisions to respective processing modules
Error Handling	If no rule matches the input set, the engine applies a default fallback rule: Three-Tier Architecture for final-year projects; MVC for mini projects. Planning default: equal phase distribution across 5 SDLC phases.



SOFTWARE USED :- MODELIO

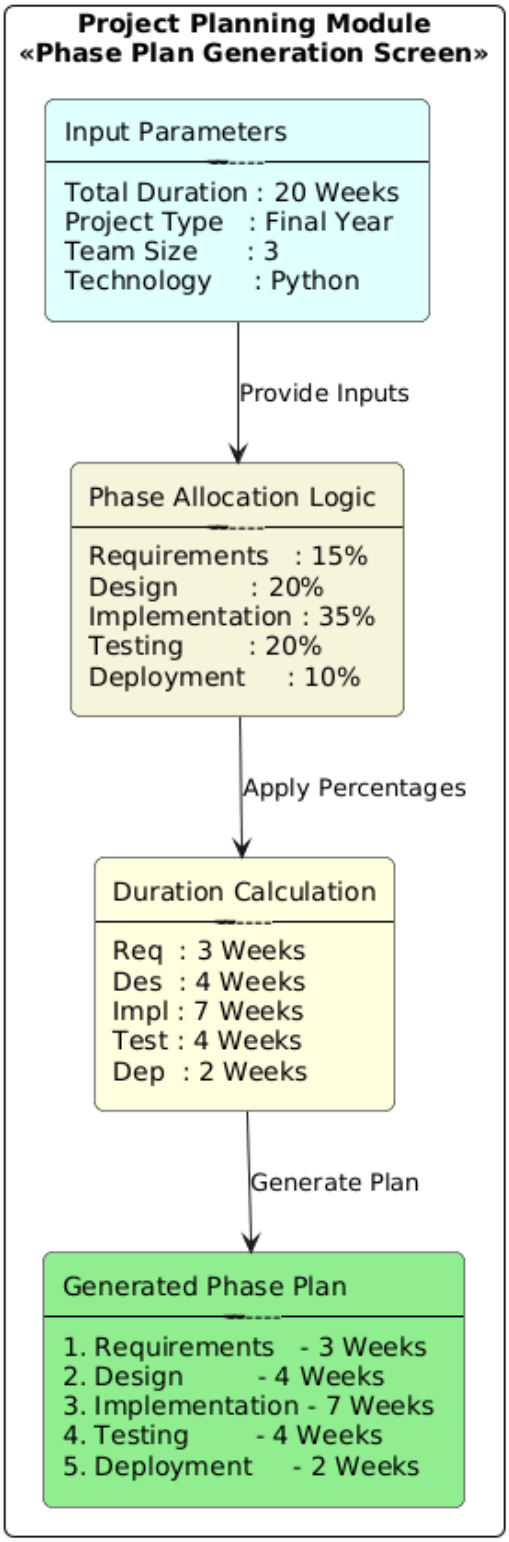


SOFTWARE USED :- MODELIO

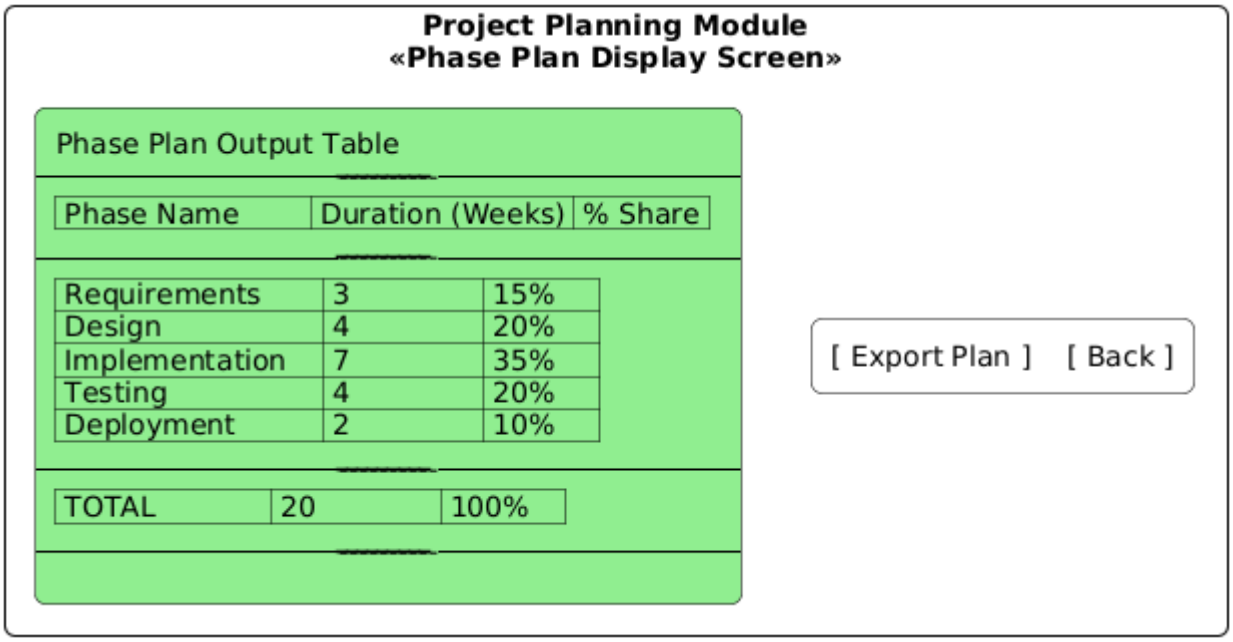
4.1.4 Project Planning Module Description

The Project Planning Module (MOD-04) receives the planning decision from the Rule Engine and generates a complete, structured, phase-wise project plan. It calculates the number of weeks allocated to each SDLC phase based on the percentage allocations specified in the matching planning rule, ensures that duration totals are correct, and writes the completed plan object to the Plan Store.

Module Name	Project Planning Module (MOD-04)
Layer	Application Layer
Interface Type	Service Interface — Synchronous (Plan Generation)
Input Received	Planning Decision from MOD-03: {rule_id, phase_allocations: [{phase_name, percentage}]}, plus duration_weeks from validated inputs
Output Produced	ProjectPlan object written to Plan Store: {plan_id, project_type, total_duration, phases: [Phase], generated_at}
Use Cases Supported	UC-04: Generate Project Plan
Processing Logic	1. Receive phase allocation percentages and total duration 2. For each phase: allocated_weeks = round(pct × total_weeks / 100, 1) 3. Validate sum of allocated weeks equals total_weeks (apply rounding correction if needed) 4. Build Phase objects with name, duration, percentage, and description 5. Generate unique plan_id (UUID) and timestamp 6. Write completed ProjectPlan object to Plan Store
SDLC Phases (Mini Project)	Requirements Analysis | System Design | Implementation | Testing | Deployment
SDLC Phases (Final-Year Project)	Requirements Analysis | System Design | Implementation | Testing | Documentation | Deployment



SOFTWARE USED :- MODELIO



SOFTWARE USED :- MODELIO

4.1.5 Architecture Recommendation Module Description

The Architecture Recommendation Module (MOD-05) receives the architecture decision from the Rule Engine and constructs a complete architecture recommendation object. It enriches the rule-based type decision with a comprehensive description, a list of key system components, and a rationale explaining why this architecture was recommended for the described project. The completed recommendation is written to the Architecture Store.

Module Name	Architecture Recommendation Module (MOD-05)
Layer	Application Layer
Interface Type	Service Interface — Synchronous (Architecture Generation)
Input Received	Architecture Decision from MOD-03: {rule_id, architecture_type, rationale}
Output Produced	ArchitectureDesign object written to Architecture Store: {architecture_id, architecture_type, description, components: [String], rationale}
Use Cases Supported	UC-03: Generate Architecture Design
Supported Architecture Types	• Monolithic Architecture — for small teams (≤2) with desktop/embedded projects • MVC Architecture — for small web teams (≤3 members) • Three-Tier Architecture — for medium-large web/mobile teams (≥4 members) • Pipeline Architecture — for data science projects with large teams • Three-Tier with API Layer — for final-year projects with ≥4 members

Processing Logic	1. Receive architecture_type from MOD-03 decision 2. Look up architecture description template for the given type 3. Build component list appropriate for the architecture type 4. Assemble full rationale combining rule-provided rationale and description 5. Generate unique architecture_id (UUID) and timestamp 6. Write ArchitectureDesign object to Architecture Store

4.2 Process Interface

The process interface section describes the stimulus-response behaviour of each system process from the user's perspective. Each process description specifies the triggering event (stimulus), the system's processing response, and the resulting output or state change.

4.2.1 Project Input Process Description

The Project Input Process handles all student interactions with the input screen, including field entry, form submission, and error correction. It is the entry point for all system processing.

User / Module Action	**System Response**
Student opens the application	System initialises to Idle state. Displays input form with four empty labelled fields and the 'Generate Plan' submit button.
Student selects project type from dropdown	Field value is stored in the input form state. Field is highlighted as active. No immediate validation triggered.
Student enters team size (numeric)	Field value is recorded. Basic type-check may provide immediate feedback for non-numeric entry.
Student enters duration in weeks (numeric)	Field value is recorded in form state.
Student selects technology stack from dropdown	Field value is stored in form state.
Student clicks 'Generate Plan' button	System transitions from InputEntry to Validation state. All four field values are collected and submitted to MOD-02 for validation.
System detects validation errors	System transitions to ValidationError state. Error messages are displayed inline beside each invalid field. Form remains populated with entered values.
Student corrects invalid fields and resubmits	System re-executes validation on corrected inputs. Proceeds to Rule Evaluation if all checks pass.
Student clicks 'Cancel' button	System returns to Idle state. Form is cleared.

4.2.2 Rule Evaluation Process Description

The Rule Evaluation Process is a system-internal process triggered automatically upon successful input validation. It accesses the Rule Base and applies the rule matching algorithm to produce planning and architecture decisions.

User / Module Action	System Response
MOD-02 forwards validated input dictionary to MOD-03	Rule Engine receives inputs. Transitions to rule loading state. Loads planning rules from Rule Base in priority order.
Rule Engine evaluates planning rules	Each rule's condition set is evaluated against the input dictionary. The first matching rule is selected. Phase allocation percentages are extracted as the planning decision.
Rule Engine evaluates architecture rules	Each architecture rule condition set is evaluated in priority order. The first matching rule is selected. Architecture type is extracted as the architecture decision.
No matching rule found (fallback condition)	System applies the default fallback rule. Logs the fallback event. Proceeds with default allocations.
Both decisions produced successfully	Planning decision dispatched to MOD-04. Architecture decision dispatched to MOD-05. Both processes may execute in parallel.

4.2.3 Input Validation Process Description

The Input Validation Process is triggered immediately upon form submission. It executes a series of ordered checks on all input fields and returns a consolidated pass or fail decision.

User / Module Action	System Response
Form submission event received by MOD-02	Validation process begins. All four field values are read from the submitted form data.
Null/empty check executed on all fields	Each field is tested for null or empty value. Failing fields are marked and added to the error list.
Enum value check for project_type	project_type value is compared against allowed enum values {mini_project, final_year_project}. Mismatch adds an error entry.
Enum value check for technology	technology value is compared against allowed enum values. Mismatch adds an error entry.
Range check for team_size	Value is verified as integer in range [1, 10]. Out-of-range values add an error entry.

Range check for duration_weeks	Value is verified as integer in range [4, 52]. Out-of-range values add an error entry.
All checks complete — no errors found	Validation passes. Validated input dictionary is returned to MOD-01 for forwarding to MOD-03.
All checks complete — errors found	Error list is returned to MOD-01. UI enters ValidationError state. Inline error messages displayed.

4.2.4 Project Planning Process Description

The Project Planning Process generates a structured phase-wise project plan based on the planning decision received from the Rule Engine.

User / Module Action	System Response
MOD-03 dispatches planning decision to MOD-04	MOD-04 receives the planning decision with phase allocation percentages and the total duration from validated inputs.
MOD-04 calculates phase durations	For each phase: allocated_weeks = round(percentage × total_weeks / 100, 1). Sum verification performed.
MOD-04 builds Phase objects	Each Phase object is constructed with phase_name, allocated_weeks, allocation_pct, and description.
MOD-04 builds ProjectPlan object	Complete plan object assembled with plan_id, project_type, total_duration, phases list, and generated_at timestamp.
MOD-04 writes plan to Plan Store	Plan object persisted to Plan Store. plan_id returned as confirmation. MOD-06 is notified that plan data is available.

4.2.5 Architecture Generation Process Description

The Architecture Generation Process constructs a complete architecture recommendation object from the architecture decision produced by the Rule Engine.

User / Module Action	System Response
MOD-03 dispatches architecture decision to MOD-05	MOD-05 receives the architecture decision containing architecture_type and preliminary rationale.
MOD-05 retrieves architecture description template	Description text for the specified architecture_type is retrieved from the internal template store.
MOD-05 builds component list	Appropriate system components for the architecture type are selected (e.g., Presentation Layer, Business Logic Layer, Data Access Layer for Three-Tier).
MOD-05 builds ArchitectureDesign object	Complete object assembled with architecture_id, architecture_type, description, components, and rationale.
MOD-05 writes design to Architecture Store	ArchitectureDesign object persisted to Architecture Store. architecture_id returned. MOD-06 notified.

5 Detailed Design

This section provides complete detailed design specifications for all seven modules of the Academic Project Planning and Automated Architecture Design Tool. Each module's detailed design covers its class structure, data members (attributes), methods (operations), and the sequence, collaboration, and activity diagrams that govern its behaviour. All diagram placeholders reference the corresponding Modelio drawing instructions in Appendix A.

5.1 Module Detailed Design

The module detailed design section specifies the class-level design of each module, including all data members with types and access modifiers, and all public and private methods with their signatures and descriptions.

5.1.1 System Interface Provider

The System Interface Provider is the top-level orchestrator of the application. It initialises all modules, manages the application lifecycle, and coordinates the high-level flow of control from input collection through to output rendering. It is the main entry point of the Python application.

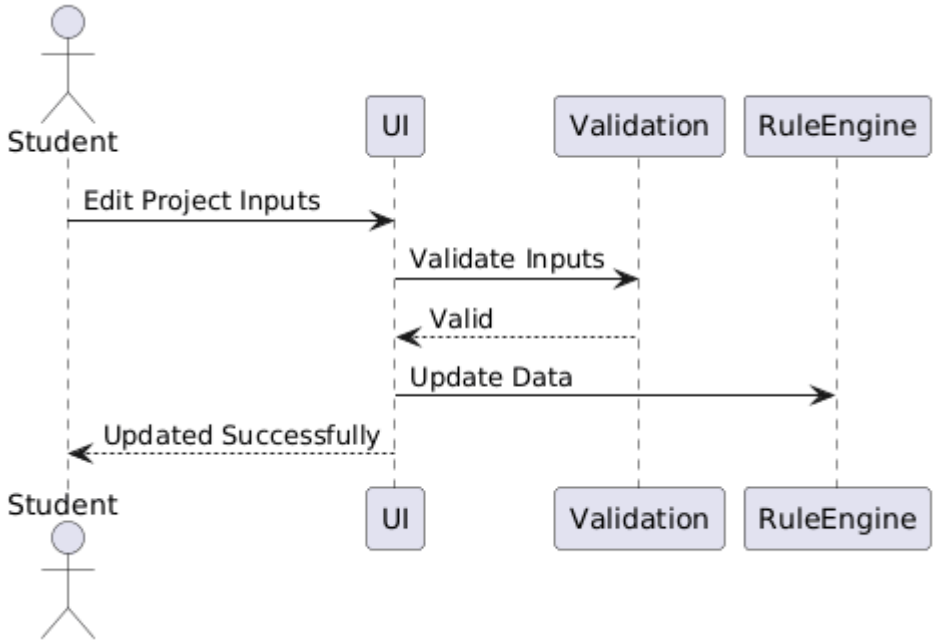
Data Members:

Attribute Name	Data Type	Access	Description
input_module	ProjectInputModule	private	Reference to the Project Input Module instance
validation_module	ValidationModule	private	Reference to the Input Validation Module instance
rule_engine	RuleEngineModule	private	Reference to the Rule Engine Module instance
planning_module	ProjectPlanningModule	private	Reference to the Project Planning Module instance
arch_module	ArchitectureModule	private	Reference to the Architecture Recommendation Module instance
output_module	OutputGenerationModule	private	Reference to the Output Generation Module instance
current_state	String	private	Tracks the current application state (Idle, InputEntry, Validation, Processing, OutputDisplay)

Methods / Operations:

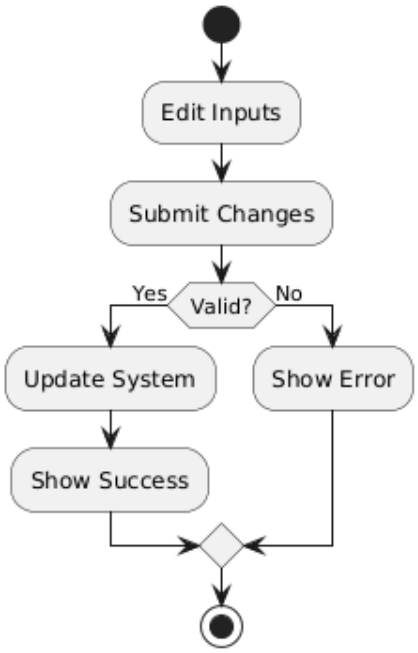
Method Name	Parameters	Return Type	Description
initialise()	None	void	Instantiates all modules, loads the Rule Base, and transitions system to Idle state
run()	None	void	Main application event loop. Listens for user events and delegates to appropriate modules
handle_submission(input_data)	dict	void	Receives form submission event. Coordinates validation → rule evaluation → generation → output
transition_state(new_state)	String	void	Updates current_state and triggers any state-entry actions
handle_modify_request()	None	void	Returns system to InputEntry state with pre-filled form data
handle_export_request()	None	void	Triggers export functionality in the Output Generation Module

SEQUENCE DIAGRAM :-



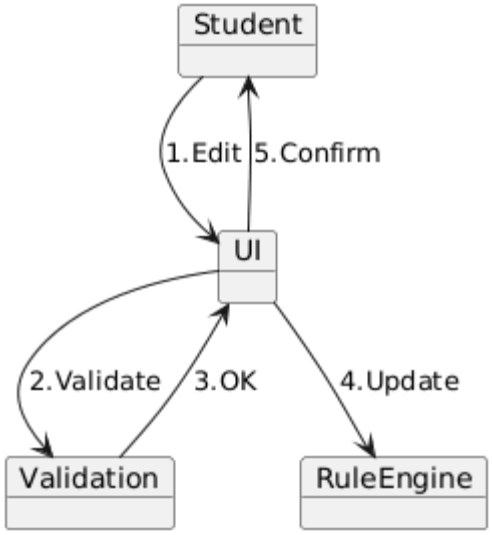
SOFTWARE USED :- MODELIO

ACTIVITY DIAGRAM :-



SOFTWARE USED :- MODELIO

COLLABRATION DIAGRAM :-



SOFTWARE USED :- MODELIO

5.1.2 Project Input Module Detailed Design

The Project Input Module implements the user interface layer, rendering the input form and output screen. It maintains the form state and communicates with higher-level modules through event callbacks.

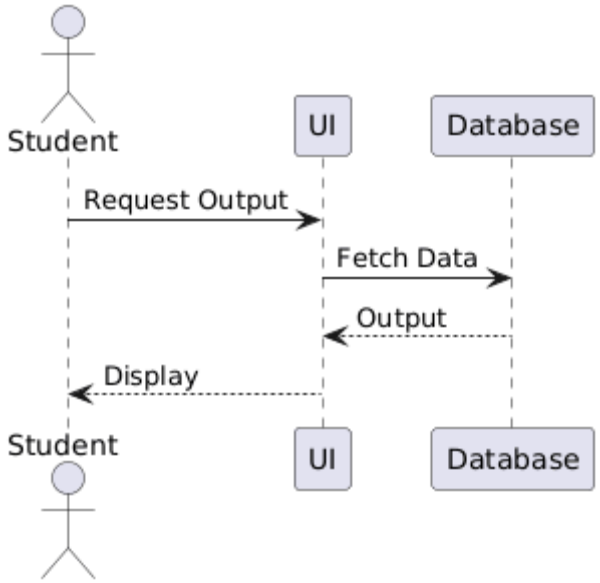
Data Members:

Attribute Name	Data Type	Access	Description
form_data	dict	private	Dictionary holding the current state of all input field values
error_list	List[dict]	private	List of validation error dictionaries returned from the Validation Module
output_data	dict	private	Formatted output data received from the Output Generation Module for rendering
current_screen	String	private	Tracks which screen is currently displayed: 'input_form' or 'output_screen'

Methods / Operations:

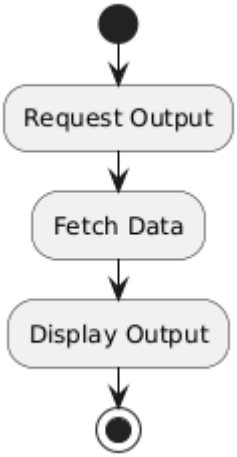
Method Name	Parameters	Return Type	Description
render_input_form(prefill_data)	dict or None	void	Renders the input form UI. If prefill_data is provided, pre-fills fields with existing values (for Modify Inputs use case)
collect_input()	None	dict	Reads and returns all current field values from the form state as a dictionary
display_validation_errors(error_list)	List[dict]	void	Renders inline error messages adjacent to the relevant input fields
render_output_screen(output_data)	dict	void	Renders the formatted project plan and architecture recommendation on the output screen
clear_errors()	None	void	Removes all displayed error messages from the input form
handle_submit_event()	None	void	Triggered by button click. Collects form data and passes to SystemProvider for processing
handle_modify_event()	None	void	Triggered by 'Modify Inputs' button click. Requests return to input form from SystemProvider

SEQUENCE DIAGRAM :-



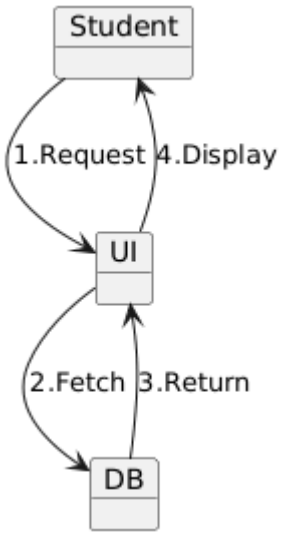
SOFTWARE USED :- MODELIO

ACTIVITY DIAGRAM :-



SOFTWARE USED :- MODELIO

COLLABRATION DIAGRAM :-



SOFTWARE USED :- MODELIO

5.1.3 Input Validation Module Detailed Design

The Input Validation Module implements all field-level and cross-field validation logic. It is designed to be entirely stateless — each call to validate() processes a fresh input dictionary and returns a new result without side effects.

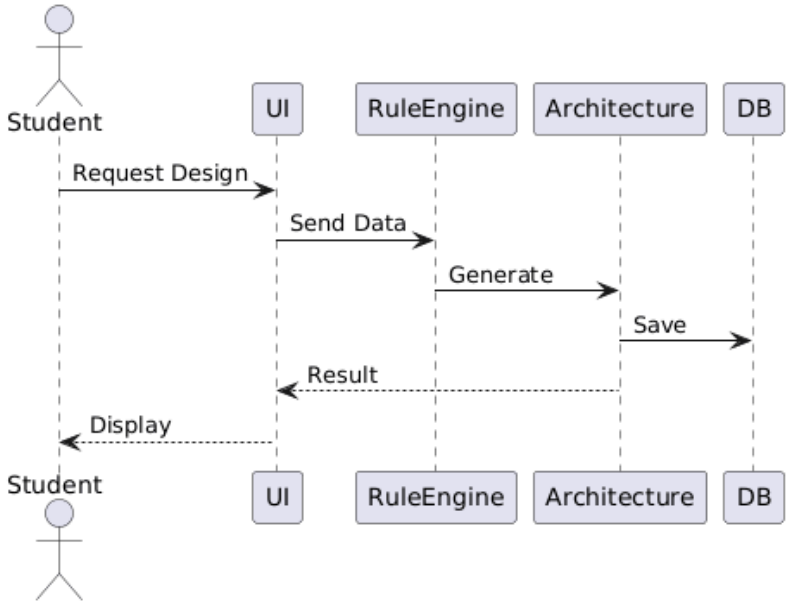
Data Members:

Attribute Name	Data Type	Access	Description
VALID_PROJECT_TYPES	List[String]	class constant	['mini_project', 'final_year_project'] — valid enum values for project_type
VALID_TECHNOLOGIES	List[String]	class constant	['web', 'mobile', 'desktop', 'data_science', 'embedded'] — valid enum values for technology
TEAM_SIZE_RANGE	Tuple(int, int)	class constant	(1, 10) — minimum and maximum acceptable team size
DURATION_RANGE	Tuple(int, int)	class constant	(4, 52) — minimum and maximum acceptable project duration in weeks

Methods / Operations:

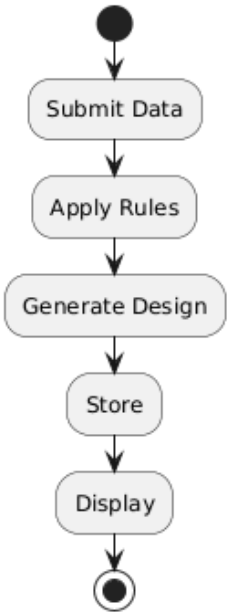
Method Name	Parameters	Return Type	Description
validate(input_data)	dict	ValidationResult	Main public interface. Executes all validation checks and returns a ValidationResult (valid=True/False, errors=list)
check_null_fields(input_data)	dict	List[dict]	Checks all required fields for null or empty values. Returns list of error entries for failing fields
check_enum_values(input_data)	dict	List[dict]	Validates project_type and technology against their respective enum sets
check_numeric_ranges(input_data)	dict	List[dict]	Validates team_size and duration_weeks against defined integer ranges
build_error_entry(field, message)	str, str	dict	Constructs a standardised error dictionary: {field: field_name, message: error_message}

SEQUENCE DIAGRAM :-



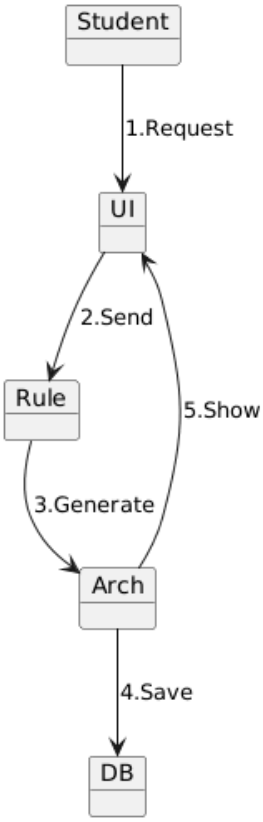
SOFTWARE USED :- MODELIO

ACTIVITY DIAGRAM :-



SOFTWARE USED :- MODELIO

COLLABRATION DIAGRAM :-



SOFTWARE USED :- MODELIO

5.1.4 Rule Engine Module Detailed Design

The Rule Engine Module implements the priority-ordered rule matching algorithm for both planning and architecture decisions. It maintains a reference to the Rule Base and evaluates rules in priority order, returning the first matched result for each decision type.

Data Members:

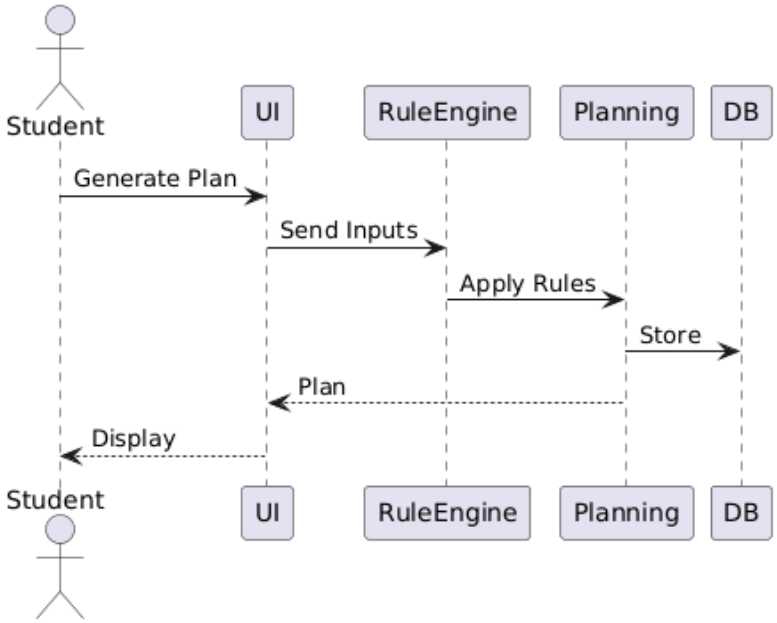
Attribute Name	Data Type	Access	Description
rule_base	RuleBase	private	Reference to the Rule Base data store providing access to all planning and architecture rules
planning_rules	List[Rule]	private	Cached list of planning rules loaded from the Rule Base, ordered by priority ascending
architecture_rules	List[Rule]	private	Cached list of architecture rules loaded from the Rule Base, ordered by priority ascending
matched_planning_rule	Rule or None	private	The planning rule that matched the current input set. None if using fallback.
matched_arch_rule	Rule or None	private	The architecture rule that matched the current input set. None if using fallback.

Methods / Operations:

Method Name	Parameters	Return Type	Description
load_rules()	None	void	Loads and caches all planning and architecture rules from the Rule Base in priority order
evaluate_planning_rules(inputs)	dict	PlanningDecision	Iterates planning_rules list and returns the action of the first rule whose conditions match the inputs
evaluate_architecture_rules(inputs)	dict	ArchitectureDecision	Iterates architecture_rules list and returns the action of the first matching rule
match_rule(rule, inputs)	Rule, dict	bool	Evaluates all conditions of a single rule against the input dictionary.

			Returns True if all conditions are satisfied.
get_fallback_planning_decision(inputs)	dict	PlanningDecision	Returns default phase allocation when no planning rule matches the input set
get_fallback_arch_decision(inputs)	dict	ArchitectureDecision	Returns default architecture recommendation when no architecture rule matches

SEQUENCE DIAGRAM:-

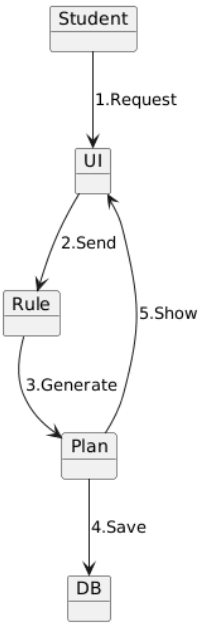


SOFTWARE USED :- MODELIO

ACTIVITY DIAGRAM:-

SOFTWARE USED :- MODELIO

COLLABRATION DIAGRAM :-



SOFTWARE USED :- MODELIO

5.1.5 Architecture Recommendation Module Detailed Design

The Architecture Recommendation Module generates a rich, structured architecture recommendation object by enriching the rule engine's decision with descriptive content, component lists, and contextual rationale sourced from internal architecture description templates.

Data Members:

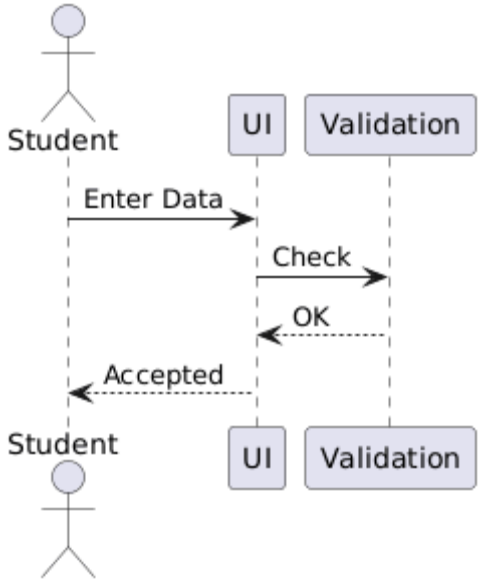
Attribute Name	Data Type	Access	Description
arch_templates	dict	private	Dictionary mapping architecture_type strings to ArchitectureTemplate objects containing description, components, and rationale templates
architecture_store	ArchitectureStore	private	Reference to the Architecture Store for persisting generated recommendations
current_recommendation	ArchitectureDesign or None	private	The most recently generated architecture recommendation object

Methods / Operations:

Method Name	Parameters	Return Type	Description

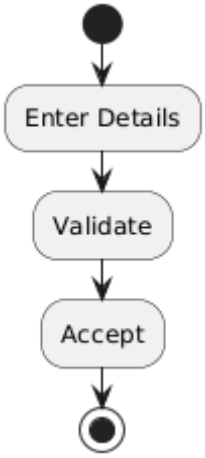
generate_recommendation(arch_decision)	ArchitectureDecision	ArchitectureDesign	Main entry point. Builds and returns a complete ArchitectureDesign object for the given decision
get_description(arch_type)	String	String	Retrieves the full description text for the specified architecture type from templates
get_components(arch_type)	String	List[String]	Retrieves the component list for the specified architecture type
build_rationale(arch_decision)	ArchitectureDecision	String	Builds the full rationale string combining the rule-provided rationale with template text
store_recommendation(design)	ArchitectureDesign	String	Writes the design object to the Architecture Store. Returns the generated architecture_id.

SEQUENCE DIAGRAM :-

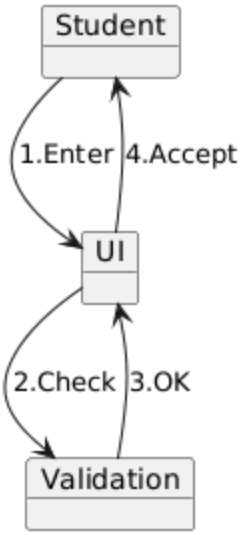


SOFTWARE USED :- MODELIO

ACTIVITY DIAGRAM :-



SOFTWARE USED :- MODELIO
COLLABRATION DIAGRAM :-



SOFTWARE USED :- MODELIO

5.1.6 Output Generation Module Detailed Design

The Output Generation Module retrieves both the generated project plan and architecture recommendation from their respective stores, formats them into a unified display structure, and delivers the final output to the Presentation Layer for rendering. It also handles the Export Project Plan use case by serialising output data to external file formats.

Data Members:

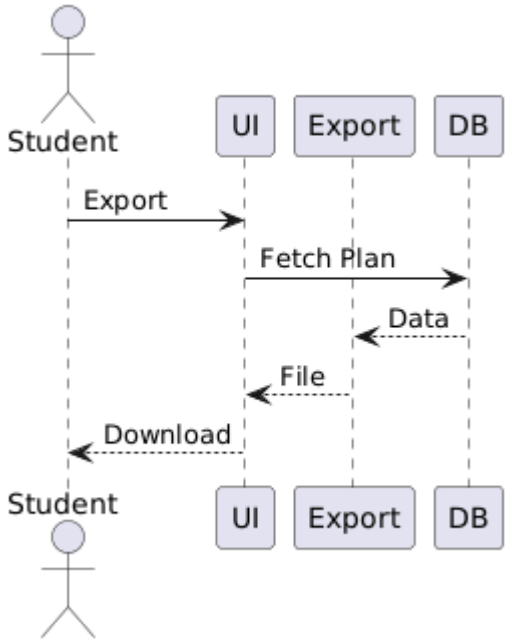
Attribute Name	Data Type	Access	Description
plan_store	PlanStore	private	Reference to the Plan Store for retrieving generated project plan data

architecture_store	ArchitectureStore	private	Reference to the Architecture Store for retrieving generated architecture recommendation data
current_output	dict or None	private	The most recently compiled output dictionary ready for rendering

Methods / Operations:

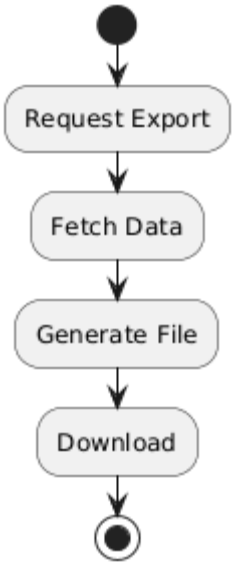
Method Name	Parameters	Return Type	Description
compile_output(plan_id, arch_id)	String, String	dict	Retrieves plan and architecture data from stores, assembles and returns a unified output dictionary
format_plan_table(plan)	ProjectPlan	List[dict]	Formats the project plan into a list of row dictionaries suitable for table rendering
format_architecture_panel(design)	ArchitectureDesign	dict	Formats the architecture recommendation into a structured panel dictionary for display
render_output(formatted_output)	dict	void	Passes the formatted output to the Presentation Layer (ProjectInputModule) for screen rendering
export_to_file(output, file_format)	dict, String	bool	Serialises the compiled output to the specified file format (PDF or TXT). Returns success status.
view_output()	None	dict	Returns the current output dictionary for display. Supports UC-02: View Output.

SEQUENCE DIAGRAM:-



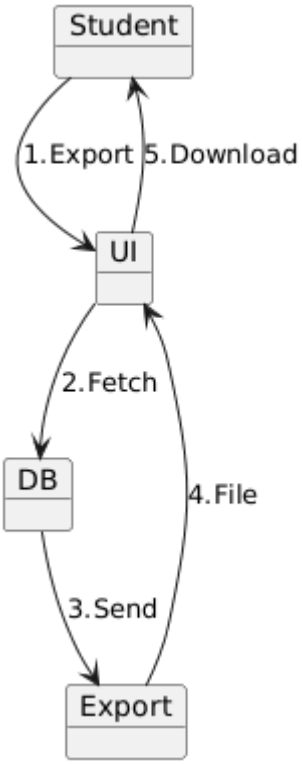
SOFTWARE USED :- MODELIO

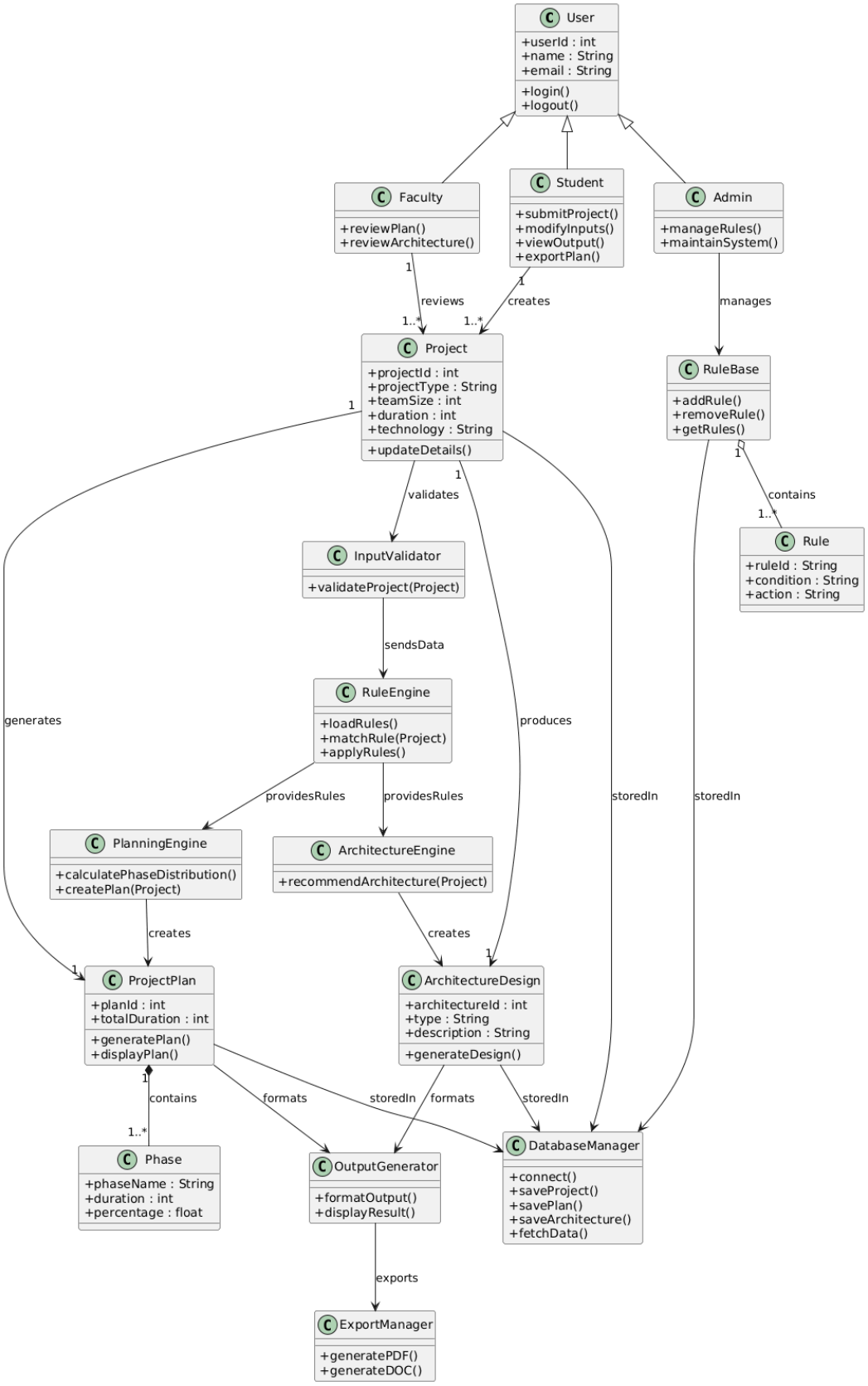
ACTIVITY DIAGRAM:-



SOFTWARE USED :- MODELIO

COLLABRATION DIAGRAM :-



SOFTWARE USED :- MODELIO

COMBINED USE CASE CLASS DIAGRAM :-